14강. TypeScript + React

TypeScript

TypeScript(타입스크립트)

- **TypeScript란?**

TypeScript는 자바스크립트를 기반으로 한 **정적 타입 언어**로, Microsoft에서 개발했습니다.

쉽게 말하면 **JavaScript + 타입 기능**이라고 보면 됩니다.

- 타입 검사 (Type Checking)
- 최신 문법 지원 (ES6+)
- 더 강력한 개발 도구 지원 (자동완성, 오류 탐지)

작성한 TypeScript 코드는 결국 **JavaScript로 변환(컴파일)** 되어 실행됩니다.

TypeScript(타입스크립트)

- **사용된 변수 선언 코드**

```
let name: string = “장그래";
let age: number = 30;
let isStudent: boolean = false;
```

- **현재 실무 현황**

• React 프로젝트 (특히 협업)
• 대규모 웹 서비스
• 유지보수가 중요한 프로젝트

요즘은 **React + TypeScript 조합이 거의 표준입니다.

TypeScript(타입스크립트)

- **설치 및 버전**

```
npm install –g typescript (전역 설치)

tsc –v    (typescript compiler)
```

- **파일 생성 및 실행**

```
hello.ts    (확장자: .ts)

tsc hello.ts    (컴파일 -> hello.js 생성됨)

node hello.js (실행)
```

TypeScript(타입스크립트)

- **기본 타입 예제 – basic.ts**

```
// string - 문자열
let message: string = "Hello, TypeScript!";
console.log(message);

// number - 숫자
let age: number = 30;
console.log(`나이: ${age}`);

// boolean - 불리언(참/거짓)
let isStudent: boolean = true;
console.log(`학생인가요? ${isStudent}`);

// 배열 - Array
let carts: string[] = ["사과", "바나나", "오렌지"];
console.log(`장바구니: ${carts.join(", ")}`);

let carts2: Array<string> = ["사과", "바나나", "오렌지"];
console.log(`장바구니 2: ${carts2.join(", ")}`);
```

TypeScript(타입스크립트)

- **기본 타입 – any**

  any는 이름 그대로 **어떤 타입의 값도 담을 수 있는** 타입입니다.

  • **할당 제한 없음:** 숫자, 문자열, 배열, 객체, 함수 등 무엇이든 대입할 수 있습니다.

  •**타입을 도저히 알 수 없는 동적 콘텐츠:** 서드파티 라이브러리에서 오는 복잡한 데이터나 구조를 알 수 없는 값을 다룰 때 사용합니다.

```
// any - 모든 타입 허용 (사용 자제)
let value: any = 10;

value = "Hello";     // OK
value = true;        // OK

console.log(`value = ${value}`);
```

TypeScript(타입스크립트)

- **interface 란?**

TypeScript의 interface는 **객체의 구조(형태)를 정의하는 설계도**입니다.

쉽게 말하면 "이 객체는 어떤 속성과 타입을 가져야 한다"를 미리 정해놓는 것.

```
// 자바스크립트 객체
let person = {
    name: "홍길동",
    age: 25,
};
```

```
// 타입 스크립트의 객체
interface Person {
    name: string;
    age: number;
}

let person2: Person = {
    name: "홍길동",
    age: 25,
};
```

TypeScript(타입스크립트)

- **interface 예제 – interface_ex.ts**

```typescript
interface Person {
    name: string;
    age: number;
}

let person2: Person = {
    name: "홍길동",
    age: 25, // age 속성이 누락되면 오류 발생
};

let person3: Person = {
    name: "우영우",
    age: 30,
};

// 출력
console.log(`이름: ${person2.name}, 나이: ${person2.age}`);
console.log(`이름: ${person3.name}, 나이: ${person3.age}`);
```

TypeScript(타입스크립트)

- **interface 예제 – interface_ex.ts**

```typescript
// readonly 속성은 객체가 생성된 후에는 변경할 수 없는 속성
// 선택적 속성(?은 객체에 있을 수도 있고 없을 수도 있는 속성
interface User {
    readonly id: number;
    name: string;
    age?: number; // age는 선택적 속성
}

let user1: User = {id: 1, name: "김철수"};

let user2: User = {id: 2, name: "이영희", age: 28};

console.log(`이름: ${user1.name}`);

user2.name = "박상희"; // name 속성은 변경 가능
console.log(`이름: ${user2.name}, 나이: ${user2.age}`);

user1.id = 3; // 오류: id는 readonly 속성이므로 변경할 수 없습니다.
```

TypeScript(타입스크립트)

- **type 이란?**

TypeScript의 `type`은 **데이터의 형태를 정의하는 또 다른 방법**입니다. `interface`와 비슷하지만, **더 유연하고 다양한 타입을 표현할 수 있는 것이 핵심**입니다.

```
type Member = {
  id: number;
  name: string;
  age: number;
};
```

TypeScript(타입스크립트)

- **type 예제 – type_ex.ts**

```typescript
// Type Alias - 타입 별칭
type Member = {
  id: number;
  name: string;
  age: number;
};

const member1: Member = {
  id: 1,
  name: "우영우",
  age: 30,
};

console.log(`회원1: ${member1.name}, 나이: ${member1.age}`);
```

TypeScript(타입스크립트)

- **type 예제 – type_ex.ts**

```typescript
// Union Type(유니언 타입)
type MemberRole = "admin" | "user";

type Member = {
  id: number;
  name: string;
  age: number;
  role: MemberRole; // MemberRole 참조
};

const member1: Member = {
  id: 1,
  name: "우영우",
  age: 30,
  role: "user"
};
```

TypeScript(타입스크립트)

- **type 예제 – type_ex.ts**

```
const member2: Member = {
  id: 2,
  name: "김철수",
  age: 25,
  role: "admin"
};

console.log(`회원1: ${member1.name}, 나이: ${member1.age}, \
권한: ${member1.role}`);

console.log(`회원2: ${member2.name}, 나이: ${member2.age}, \
권한: ${member2.role}`);
```

TypeScript(타입스크립트)

- **interface vs type 포함 예제 – multi_ex.ts**

```typescript
// 학점 정의
type Grade = "A" | "B" | "C" | "D" | "F";

// 과목 정의
interface Course {
    courseId: number;
    courseName: string;
    grade?: Grade; // 선택적 속성
}

// 학생 정의
interface Student {
    id: number;
    name: string;
    subjects: Course[]; // 학생이 수강하는 과목 목록
}
```

TypeScript(타입스크립트)

- **interface vs type 포함 예제 – multi_ex.ts**

```
// 학생 객체 생성
let student1: Student = {
    id: 1,
    name: "김선화",
    subjects: [
        { courseId: 101, courseName: "수학", grade: "A" },
        { courseId: 102, courseName: "영어", grade: "B" },
    ]
};

// 학생 정보 출력
console.log(`학생명: ${student1.name}`);
student1.subjects.forEach(subject => {
    console.log(`과목명: ${subject.courseName}, 학점: ${subject.grade}`);
});
```

TypeScript(타입스크립트)

- **interface vs type 차이점 – multi_ex.ts**

```typescript
interface Point {
    x: number;
    y: number;
}

type PointType = {
    x: number;
    y: number;
};

const point1: Point = { x: 10, y: 20 };
const point2: PointType = { x: 30, y: 40 };

console.log(`Point 인터페이스: (${point1.x}, ${point1.y})`);
console.log(`Point 타입 별칭: (${point2.x}, ${point2.y})`);
```

TypeScript(타입스크립트)

- **함수의 타입 예제 – function_ex.ts**
  - return 이 있는 함수

```
/*function add(x: number, y: number): number {
  return x + y;
}*/

const add = (x: number, y: number): number => {
  return x + y;
}
const sum = add(5, 10);
console.log(`합: ${sum}`);
```

TypeScript(타입스크립트)

- **함수의 타입 예제 – function_ex.ts**
  - return 이 없는 함수 – void를 명시

```
/*function greet(name: string): void {
  console.log(`Hello, ${name}!`);
}*/

const greet = (name: string): void => {
  console.log(`Hello, ${name}!`);
}
greet("TypeScript");
```

TypeScript(타입스크립트)

- **함수의 타입 예제 – function_ex.ts**
  - 객체를 반환하는 함수

```
const createUser = (id: number, name: string): { id: number; name: string } => {
  return { id, name };
}

// createUser 함수는 id와 name을 받아서 User 객체를 반환합니다.
const user1 = createUser(1, "Alice");
console.log(`사용자 ID: ${user1.id}, 이름: ${user1.name}`);
```

TypeScript(타입스크립트)

- **함수의 타입 예제 – function_ex.ts**
  - 함수 타입을 변수에 할당

```
type MathOperation = (a: number, b: number) => number;

const multiply: MathOperation = (a, b) => a * b;
const value1 = multiply(5, 10);
console.log(`곱: ${value1}`);

const divide: MathOperation = (a, b) => a / b;
const value2 = divide(10000, 2.5);
console.log(`나눗셈: ${value2}`);
```

TypeScript(타입스크립트)

- **Record<K, T>란?**

  TypeScript에서 **Record<Keys, Type>**는 객체의 **키(Key)**와 **값(Value)**의 타입을 간편하게 정의할 수 있게 도와주는 유틸리티 타입입니다.

  - K (Keys): 객체의 속성 키로 사용할 타입 (보통 문자열 리터럴의 연합(Union)이나 string, number 등이 옵니다).
  - T (Type): 각 키에 할당될 값의 타입입니다.

```typescript
type StringMap = Record<string, string>;


const translations: StringMap = {
  hello: '안녕하세요',
  goodbye: '안녕히 가세요',
};
```

TypeScript(타입스크립트)

- **Record<K, T> - record_ex.tsx**

```typescript
// 동적 키와 값
type StringMap = Record<string, string>;

const translations: StringMap = {
  hello: '안녕하세요',
  goodbye: '안녕히 가세요',
};

// 출력
console.log(translations.hello); // '안녕하세요'
console.log(translations.goodbye); // '안녕히 가세요'
```

TypeScript(타입스크립트)

- **Record<K, T> - record_ex.tsx**

```typescript
// 특정 키만 허용
type Fruit = 'apple' | 'banana' | 'orange';

const prices: Record<Fruit, number> = {
  apple: 1000,
  banana: 1500,
  orange: 2000,
};

// 출력
for (const fruit in prices) {
  console.log(`${fruit}: ${prices[fruit as Fruit]}원`);
}
```

React + TypeScript

- **React 개발환경**

도구	설명
Node.js	TypeScript 실행 환경
npm	패키지 관리
Vite	빠른 React 개발 환경 (CRA 방식은 로딩 속도가 느려서 2025년부터 중지됨)

React + TypeScript

- **프로젝트 만들기**

```
npm create vite@latest my-app

framework – React

language - Typescript
```

- **깃허브 리포지터리 생성 : react-typscript**

React + TypeScript

- **README.md**

```md
## TypeScript(타입 스크립트) 언어 배우기
### 1. TypeScript 언어 개요
### 2. 설치 및 실행
### 3. 기초 문법
---------------------------------------------------

## TypeScript 언어 기반의 React 프로젝트
### my-app 프로젝트 생성
#### 설치
- npm create vite@latest my-app
  1. framework: React
  2. language: TypeScript
```

React + TypeScript

- **main.tsx**

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

React + TypeScript

- **App.tsx**

```
function App() {

  return (
    <>
      <section id="center">
        <h2>TypeScript 기반 React 프로젝트</h2>
        <Greet name="React" />
      </section>
    </>
  )
}

export default App
```

React + TypeScript

- **Props로 데이터 전달**

TypeScript 기반 React 프로젝트

Hello, React!

Player

이름: 신유빈

나이: 22

클릭

React + TypeScript

- **Props 타입 – Greet.tsx**

```
// Props의 타입 정의
const Greet = ({name}: {name: string}) => {
  return <h2>Hello, {name}!</h2>
}

export default Greet
```

React + TypeScript

- **Props 타입 – App.tsx**

```
function App() {
  const playerObj = {
    name: '신유빈',
    age: 22,
  }

  const clickHandler = () => {
    alert('버튼이 클릭되었습니다.')
  }

  return (
    <>
      <section id="center">
        <h2>TypeScript 기반 React 프로젝트</h2>
        <Greet name="React" />
        <Player player={playerObj} clickHandler={clickHandler} />
      </section>
    </>
  )
}
```

React + TypeScript

- **Props 타입 – Player.tsx**

```
const Player = ({ player, clickHandler }: {
  player: { name: string; age: number };
  clickHandler: () => void}) => {

  return (
    <div>
      <h1>Player</h1>
      <p>이름: {player.name}</p>
      <p>나이: {player.age}</p>
      <button onClick={clickHandler}>클릭</button>
    </div>
  )
}

export default Player
```

React + TypeScript

- **Props 타입 – Player2.tsx**

```
// type - 객체의 구조를 정의하는 타입
type PlayerProps = {
  player: {
    name: string
    age: number
  }
  clickHandler: () => void
}

const Player2 = ({ player, clickHandler }: PlayerProps) => {
  return (
    <div>
      <h1>Player2</h1>
      <p>이름: {player.name}</p>
      <p>나이: {player.age}</p>
      <button onClick={clickHandler}>클릭</button>
    </div>
  )
}
```

React + TypeScript

- **Counter**

React + TypeScript

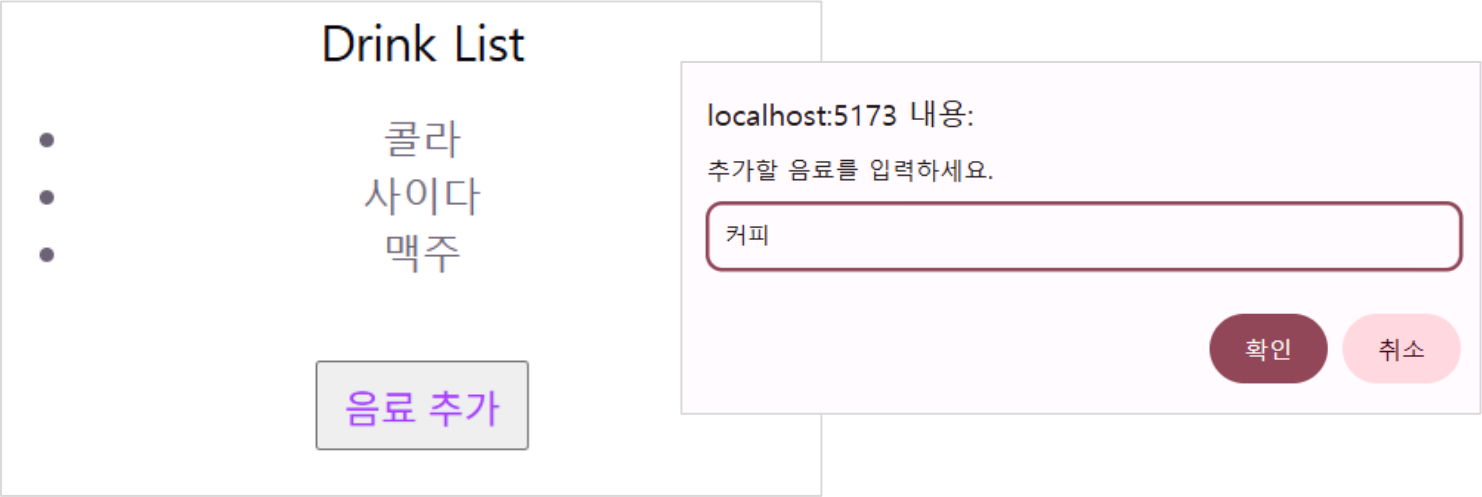
- **제네릭 타입 - <number>**

```
const Counter = () => {
  //<number>은 count가 number 타입임을 명시적으로 지정하는 제네릭 문법
  const [count, setCount] = useState<number>(0)

  return (
    <div>
      <h3>Counter</h3>
      <p>현재 카운트: {count}</p>
      <button onClick={() => setCount(count + 1)}>증가</button>
      <button onClick={() => setCount(count - 1)}>감소</button>
      <button onClick={() => setCount(0)}>초기화</button>
    </div>
  )
}
```

React + TypeScript

- **음료 리스트**



React + TypeScript

- **제네릭 배열 타입 - <string[]>**

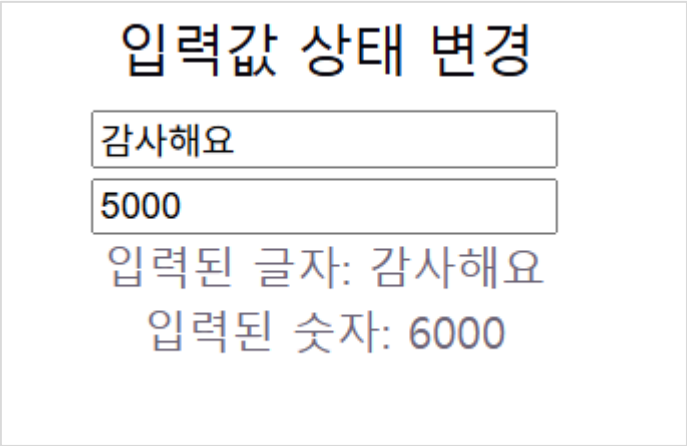
```
const DrinkList = () => {
  const [drinks, setDrinks] = useState<string[]>(['콜라', '사이다', '맥주'])

  // 음료 추가
  const addDrink = () => {
    const newDrink = prompt('추가할 음료를 입력하세요.')
    if (newDrink) {
      setDrinks([...drinks, newDrink])
    }
  }

  return (
    <div>
      <h2>Drink List</h2>
      <ul>
        {drinks.map((drink, index) => (
          <li key={index}>{drink}</li>
        ))}
      </ul>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  )
}
```

React + TypeScript

- **이벤트 객체의 타입** - `React.ChangeEvent<HTMLInputElement>`



React + TypeScript

- **이벤트 객체의 타입** - `React.ChangeEvent<HTMLInputElement>`

```
const InputValueState = () => {
  const [text, setText] = useState<string>('');
  const [num, setNum] = useState<number>(0);

  // 문자 입력값 상태 변경
  // 이벤트 객체의 타입은 React.ChangeEvent<HTMLInputElement>입니다.
  const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    console.log(e.target.value);
    setText(e.target.value)
  }

  // 숫자 입력값 상태 변경
  const handleNumChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setNum(Number(e.target.value))
  }
```

React + TypeScript

- **이벤트 객체의 타입** - `React.ChangeEvent<HTMLInputElement>`

```
return (
  <div>
    <h2>입력값 상태 변경</h2>
    <input
      type="text"
      value={text}
      onChange={handleTextChange}
      placeholder='글자를 입력하세요'
    />
    <br />
    <input
      type="number"
      value={num}
      onChange={handleNumChange}
      placeholder='숫자를 입력하세요'
    />
    <p>입력된 글자: {text}</p>
    <p>입력된 숫자: {num + 1000}</p>
  </div>
)
}
```

할 일 관리 앱

To-do List

할 일을 입력하세요. 추가

- [x] ~~리액트 공부하기~~
- [ ] 타입스크립트 배우기
- [ ] 환절기 감기 조심하기

할 일 관리 앱

- **Todos.tsx**

```
// Todo 타입 정의
export type Todo = {
  id: number
  text: string
  completed: boolean
}

const Todos = () => {
  const [todos, setTodos] = useState<Todo[]>([
    { id: 1, text: '리액트 공부하기', completed: false },
    { id: 2, text: '타입스크립트 배우기', completed: false },
  ])
  const [inputValue, setInputValue] = useState('')
```

할 일 관리 앱

- **Todos.tsx**

```
// 할 일 추가 함수
const addTodo = () => {
  if (!inputValue.trim()) return // 빈 문자열 입력 방지

  setTodos([...todos, {
    id: Date.now(), // id 중복 방지
    text: inputValue,
    completed: false
  }])
  setInputValue('') // 입력 필드 초기화
}

// 할 일 완료 토글 함수
const toggleTodo = (id: number) => {
  setTodos(todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  ))
}
```

할 일 관리 앱

- **Todos.tsx**

```
return (
  <div className="todo-container">
    <h2>To-do List</h2>
    <input
      type="text"
      placeholder="할 일을 입력하세요."
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
    />
    <button onClick={addTodo}>추가</button>

    {/* 할 일 목록 */}
    <TodoList todos={todos} toggleTodo={toggleTodo} />
  </div>
)
}
```

할 일 관리 앱

- **TodoList.tsx**

```
import type {Todo} from './Todos'

type TodoListProps = { // TodoListProps 타입 정의
  todos: Todo[]
  toggleTodo: (id: number) => void
}

const TodoList = ({ todos, toggleTodo }: TodoListProps) => {
  return (
    <div className="todo-list">
      <ul>
        {todos.map(todo => (
          <li
            key={todo.id}
            style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}>
            <input
              type="checkbox"
              checked={todo.completed}
              onChange={() => toggleTodo(todo.id)}
            />
            {todo.text}
          </li>
        ))}
      </ul>
```

할 일 관리 앱

- App.css

```
.todo-container {
  width: 400px;
  margin: 20px auto;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.todo-list {
  margin-top: 20px;
}
.todo-list ul {
  list-style: none;
  padding: 0;
}
.todo-list li {
  display: flex;
  align-items: center;
  margin-bottom: 10px;
}
```